



UNIVERSIDAD
CATÓLICA DE
TEMUCO

Proyecto Matemática Discreta.

Teoría de Grafos aplicada.

Integrantes: Danna Ramirez, Matías Manquela, Amanda Contreras, Vicente Fonseca.

Profesor: Elliot Mardones Arias.

Asignatura: Matemática Discreta.

Fecha de Entrega: 22-06-2026.

1.- Introducción:

La búsqueda de rutas óptimas es un problema **ampliamente estudiado en matemáticas, informática y logística**. Determinar el camino más conveniente entre dos puntos permite **optimizar recursos, reducir costos y mejorar la eficiencia de sistemas de transporte, distribución y navegación**.

En este proyecto se modeló una **red de ciudades** europeas mediante un **grafo ponderado**. Cada **ciudad** fue representada como un **vértice** y cada **conexión** entre ciudades como una **arista**. Como **criterio de ponderación** se utilizó la distancia **aproximada entre ciudades expresada en kilómetros**.

El **conjunto de ciudades** seleccionado incluye **Roma, Nápoles, Perugia, Pescara, Florencia, Bari, Pisa, Bolonia, Génova, Venecia, Milán, Niza, Zúrich, Múnich y Viena**. Estas ciudades fueron escogidas debido a la existencia de conexiones terrestres que permiten construir un **grafo conexo y representativo de una red de transporte real**.

El **objetivo principal** del proyecto es diseñar e implementar una **aplicación capaz de calcular la ruta de menor distancia entre una ciudad de origen y una ciudad de destino utilizando teoría de grafos y algoritmos de camino mínimo**.

2.- Modelamiento Matemático:

2.1 Definición del Grafo.-

La red de ciudades se representa mediante un **grafo ponderado no dirigido** dado por:

$$[G = (V, E, W)] \quad \text{Donde;} \quad \begin{aligned} (V) &= \text{Conjunto de Vértices.} \\ (E) &= \text{Conjunto de Aristas.} \\ (W) &= \text{Función de Peso.} \end{aligned}$$

2.2 Conjunto de Vértices.-

El **conjunto de vértices** está formado por las siguientes **ciudades** europeas seleccionadas.

$$[(V) = \{Roma, Napoles, Perugia, Pescara, Florencia, Bari, Pisa, Bolonia, Genova, Venecia, Milan\} \\ \{Niza, Zurich, Munich, Viena\}] \quad \text{Por lo tanto } |V| = 15.$$

2.3 Conjunto de Aristas.-

Las **conexiones** entre ciudades consideradas son las **siguientes**.

$$[(E) = \{(Roma, Napoles), (Roma, Perugia), (Roma, Pescara), (Roma, Florencia), (Nápoles, Bari), \\ (Perugia, Florencia), (Pescara, Bari), (Pescara, Perugia), (Florencia, Pisa), (Florencia, Bolonia), \\ (Pisa, Genova), (Bolonia, Venecia), (Bolonia, Milan), (Genova, Milan), (Genova, Niza), \\ (Milan, Venecia), (Milán, Zúrich), (Venecia, Viena), (Zúrich, Múnich), (Múnich, Viena)\}]$$

$$\text{Por lo tanto } |E| = 20.$$

2.4 Función de Peso.-

Está definida por la función contigua.

$$W = E \rightarrow R + \quad \text{"Para cada arista } e \text{ perteneciente a } E, \\ \text{existe un valor } w(e) \text{ dado por un número real positivo"}$$

Ejemplo: $w(Roma, Nápoles) = 226$, Representando esto una distancia en KM.

2.5 Propiedad de Conexidad.-

"Para cualquier par de vértices u y v que pertenezcan al conjunto (V) existe al menos un camino P que los conecta."

$$\forall u, v \in V, \exists P(u, v)$$

2.6 Costo de un camino.-

$$C(P) = \sum_{i=0}^{k-1} w(v_i + v_{i+1})$$

Ruta Óptima.-

$$P^x = \arg \min C(P)$$

3.- Metodología

3.1 Algoritmo seleccionado.-

Para resolver el problema de camino mínimo se seleccionó el algoritmo de Dijkstra. Este algoritmo permite encontrar la distancia mínima desde un nodo origen hacia todos los demás nodos de un grafo cuyos pesos son positivos.

La elección de este algoritmo resulta adecuada debido a que todas las conexiones del proyecto representan distancias y, por lo tanto, poseen valores positivos.

3.2 Funcionamiento General.-

El algoritmo realiza los siguientes pasos:

1. Inicializar todas las distancias con infinito.
2. Asignar distancia cero al nodo de origen.
3. Seleccionar el nodo no visitado con menor distancia conocida.
4. Actualizar las distancias de sus vecinos.
5. Repetir hasta procesar todos los nodos.

3.3 Pseudocódigo.-

ALGORITMO: Dijkstra (camino mínimo desde un origen)

ENTRADA:

$G = (V, E, w)$: grafo ponderado, $w(u,v) > 0$ para toda arista

s : vértice de origen

t : vértice de destino

SALIDA:

P^* : camino de costo mínimo desde s hasta t

$C(P^*)$: costo total del camino

INICIO

1. Inicialización

PARA CADA vértice v EN V HACER

$dist[v] \leftarrow INFINITO$

$previo[v] \leftarrow NULO$

FIN PARA

$dist[s] \leftarrow 0$

$Q \leftarrow$ conjunto de todos los vértices de V // cola de prioridad

2. Bucle principal

MIENTRAS Q no este vacío HACER

$u \leftarrow$ vértice en Q con menor $dist[u]$

 QUITAR u de Q

 SI $u == t$ ENTONCES

 ROMPER // ya encontramos el destino, podemos detenernos

 FIN SI

 PARA CADA vecino v de u (tal que (u,v) pertenece a E) HACER

$alternativa \leftarrow dist[u] + w(u, v)$

 SI $alternativa < dist[v]$ ENTONCES

$dist[v] \leftarrow alternativa$

$previo[v] \leftarrow u$

 FIN SI

 FIN PARA

FIN MIENTRAS

3. Reconstruccion del camino óptimo

$P^* \leftarrow$ lista vacía

$actual \leftarrow t$

MIENTRAS $actual \neq NULO$ HACER

 INSERTAR $actual$ al inicio de P^*

$actual \leftarrow previo[actual]$

FIN MIENTRAS

$C(P^*) \leftarrow dist[t]$

DEVOLVER $P^*, C(P^*)$

FIN ALGORITMO

4.- Implementación

4.1 Estructura General.-

La aplicación fue desarrollada en Python y organizada en distintos módulos:

- datos.py
- Algoritmo.py
- mapa.py
- interfaz.py
- mapa_base.py

4.2 Representación del Grafo

La red de ciudades se almacenó utilizando una estructura de diccionarios de adyacencia. En esta representación, cada ciudad corresponde a una clave del diccionario principal y sus conexiones directas se almacenan en un diccionario secundario junto con la distancia asociada a cada conexión.

La construcción del grafo se realiza mediante la función "**Construir_grafo()**", la cual recorre todas las conexiones definidas en el conjunto de datos y agrega las aristas en ambos sentidos, ya que el grafo es del tipo no dirigido.

Un fragmento representativo de la estructura generada es el siguiente:

```
{  
  "Roma": {  
    "Nápoles": 226,  
    "Perugia": 173,  
    "Pescara": 208,  
    "Florenzia": 275  
  },  
}
```

Esta representación permite acceder de manera eficiente a todas las ciudades vecinas de un vértice y a los pesos asociados a cada conexión. Gracias a ello, el algoritmo de Dijkstra puede recorrer el grafo y actualizar las distancias mínimas con un bajo costo computacional.

Además, dado que las conexiones se agregan en ambos sentidos:

$$\begin{aligned} \text{grafo}[\text{origen}][\text{destino}] &= \text{distancia} \\ \text{grafo}[\text{destino}][\text{origen}] &= \text{distancia} \end{aligned}$$

Se garantiza que el grafo sea no dirigido, representando adecuadamente las rutas bidireccionales entre ciudades.

4.2 Comparación Algoritmo Bellman - Ford.-

Criterio	Dijkstra	Bellman-Ford
Complejidad temporal.	$O((V+E) \log V)$ con cola de prioridad	$O(V \times E)$
Pesos negativos.	No admite pesos negativos	Si admite pesos negativos
Detección de ciclos negativos.	No detecta	Si detecta
Estrategia.	Voraz (greedy): Explora siempre el vértice de menor distancia acumulada	Relaja todas las aristas $V-1$ veces iterativamente.
Optimalidad.	Garantiza solución óptima si todos los pesos son positivos	Garantiza una solución óptima incluso con pesos negativos.
Eficiencia en grafos grandes.	Alta adecuación para grafos densos y dispersos	Baja: Más lento en grafos con muchas aristas.
Aplicación en este proyecto.	SELECCIONADO: Todos los pesos son distancias positivas en km, condición que aprovecha para ser más eficiente.	DESCARTADO: No aporta ventaja adicional ya que no existen pesos negativos en el grafo constituido.

4.4 Librerías Utilizadas.-

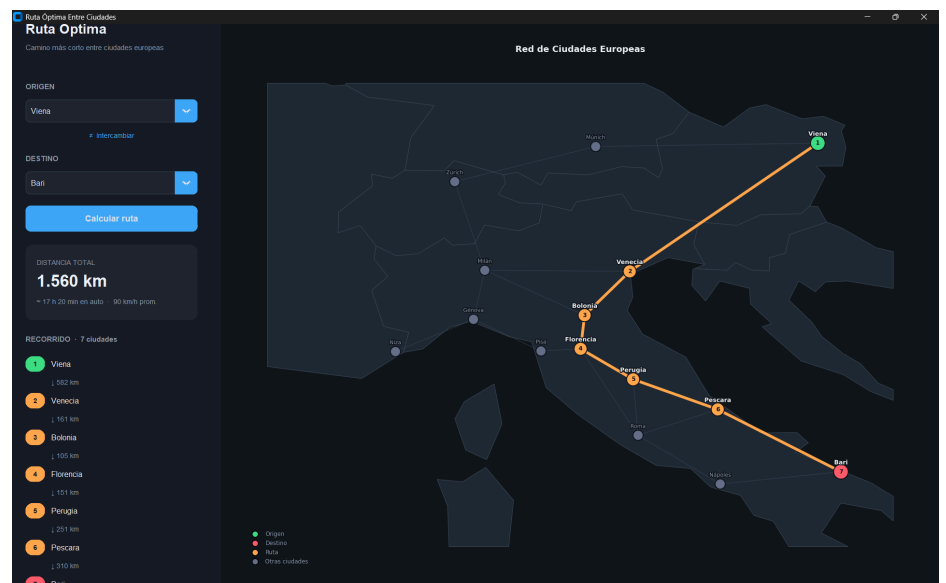
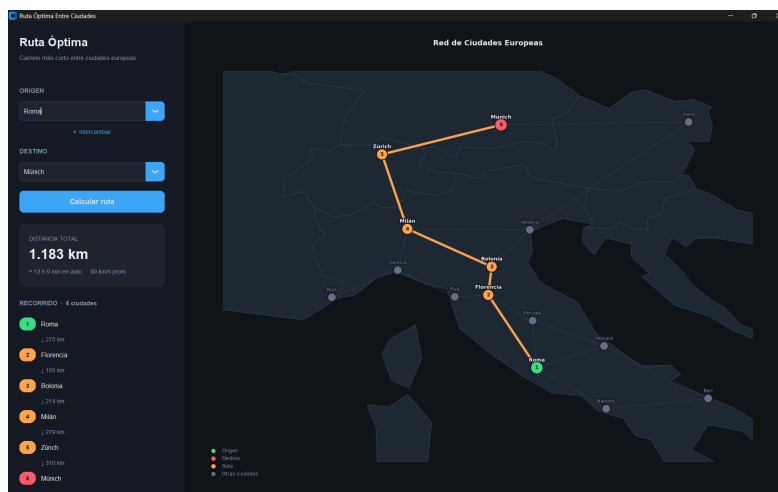
- heapq
- customtkinter
- networkx
- matplotlib

4.5 Interfaz Gráfica.-

La interfaz desarrollada permite:

- Seleccionar ciudad de origen.
- Seleccionar ciudad de destino.
- Ejecutar el cálculo de la ruta óptima.
- Visualizar el recorrido encontrado.
- Mostrar el costo total.
- Visualizar gráficamente el grafo.

5.- Pruebas del programa



6.- Conclusiones

El desarrollo de este proyecto permitió **aplicar conceptos fundamentales de teoría de grafos**, tales como vértices, aristas, pesos, caminos y conectividad.

Así mismo, se comprobó la utilidad práctica de los algoritmos de camino mínimo para **resolver problemas de optimización en redes de transporte**.

La implementación del **algoritmo de Dijkstra** demostró ser **eficiente para grafos ponderados con pesos positivos**, permitiendo obtener rutas óptimas de forma rápida y precisa.

Además, el desarrollo de una interfaz gráfica **facilitó la interacción con el usuario** y permite representar visualmente la **estructura del grafo y los recorridos obtenidos**.

Como posibles mejoras futuras se considera incorporar un **mapa geográfico real**, aumentar la cantidad de ciudades consideradas y utilizar **tiempos reales de viaje en lugar de distancias estáticas**.

6.1 Todas las fuentes en: <https://github.com/vfonseca2026-afk/ProyectoGraphos>